Stochastic Simulation Assignment 1
Limit book order problem

L. Troiakova (3377822) & W. Hui (3307530)

University of Twente
May 1, 2026

1 Introduction

In this report we describe Limit Order Book Simulation. We present our code as well as explain and interpret our results.

2 Assumptions

We assume that each arriving order's classification of limit or market is independent of the classification of side chosen. Also we assume if there is no best bid price and best ask price exist, the market order will cancel itself immediately. Fill time is only for matched limit orders, orders that are cancelled are not considered in this recording. Prices are drawn from the same sequence regardless of bid or ask side.

3 Pseudo-code

Algorithm 1 Order Arrival Event

if limit order: **then** ▷ P(limit order) = 0.7
  if prob $<$ 0.5: **then** assign order = bid
  else: assign = ask
  end if
  calculate the limit price and record **limit order**
  record the **spread** before modifying queue ▷ prevent changes affect any unrecorded spread(same reason for others)
  increment limit_counter
  append event to ask queue or bid queue
  insert new event to the queue
  record new **spread**
  schedule the **cancellation** of the event
  do **matching**
else: ▷ which means it is market order
  record as market order
  decide it is bid or ask
  if find the best opposite side **then**:
    cancel the opposite's **cancellation** event and remove the opposite from its queue
    assign opposite status = "matched"
    record time and update queue's and spread's status
    trade count += 1
    if count == 500 **then** stop and record T as current time ▷ when the 500th order is market order
    end if
  end if
end if

Algorithm 2 Cancellation Event

if order.matched **then**: skip this event ▷ only do cancellation if not matched
else:
  record spread
  delete this order from bid(or ask) queue
  mark the order status as "cancelled"
  update queue length
  count cancel counter
  go back to run **matching**
end if

Algorithm 3 Matching Mechanism

called at the end of both events
while bid queue and ask queue both non-empty **do**:
    best bid = Max(price in bid queue)
    best ask = min(price in ask queue)
    if bestBid.price $\geq$ bestAsk.price **then**:
        track spread before
        delete best bid and best ask from according queues
        cancel their **cancellation** events
        record time
        update queue status(length)
        track spread after
        count the number of trades += 1
        if trade count == 500 **then**: stop and record T as current time ▷ when the 500th order is limit order
        end if
    else: break
    end if
end while

Algorithm 4 Spread Mechanism

if bid queue and ask queue both non-empty **then**:
    record time t
    assign the current spread = bestAsk.price - bestBid.price to t
else:
    assign to None
end if

4 Cancellation-Match Interaction

For any order, the final status can only be "matched" or "cancelled", these two status are mutually exclusive."matched" is reached by matching engine and "cancelled" is reached by *event_cancel*()
When a new order arrived, it could be and only be one of the two types market order or limit order. If the order is a marker order, it is immediately matched inside *event_arrival*() and assigned its status to "matched".It will never reach its cancellation event or reach status "cancelled".
If the order is a limit order, its cancellation event is immediately scheduled and we store this status on the order object. So every order will be assigned to cancel after some time. When an order is matched, we cancel its cancellation event. The *sim.cancel*() is written inside the matching engine for both limit order and in arrival event for the opposite order of market order. And we prevent the wrong cancellation by adding a check in cancellation event. We check if the order is matched at the beginning of cancellation before calling the actual *sim.cancel*(). So the cancel-match interaction always lead to one of the two final states without crash.

5 Data collection

Data collected	Variable name in code	Event
bid-ask spread	*spread*	matching, arrival, cancellation
Time	*lob.time*	matching(for limit), arrival(for market)
Number of cancellations	*cancel_counter*	cancellation
Number of limit orders	*limit_counter*	arrival
Number of trades	*count*	matching, arrival
bid-ask queue length	*bid_length, ask_length*	matching, arrival, cancellation

TABLE 1: what data is collected and at which events

The spread is collected in every event before and after they actually modify the queue and changing other variables. We only collect time in every matching for limit orders and in arrival event for market orders. The time in

cancellation is recorded but not collected.
We collect the number of limit orders and cancellations in event_cancel and event_arrival. The number of matched orders are calculated using $\#limitorders - \#cancellations$.The counters are updated with $.increment()$.
When we remove the orders being matched or cancelled from the queue, we reduce the queue length of according queue. When an order arrived, we add the length. After every add or remove of orders we called $queue\_stauts()$ to update length at time.

6 Result:Present all four required metrics and briefly interpret them.

Metric	Result(rounded to 3 decimals)
time-averaged bid-ask spread	1.715
average time-to-fill	12.708
cancellation rate	0.291
average bid queue length	0.854
average ask queue length	1.043

TABLE 2: Result

We did all the average calculations with $.mean()$. The detailed result value is based on random seed = 42 (see Table 2). But the range of value approximately same and the interpretations and conclusions holds independent of the random seed.

6.1 Time-averaged bid-ask spread

Each time a new event arrives or gets cancelled, we record the highest bid and lowest ask and compute the spread at time t as follows: $spread(t) = best\_ask(t) - best\_bid(t)$
We also record the time when this happened. Then the time-averaged bid-ask spread is calculated by:

$$\frac{\sum_{i=0}^{n-1} s_i(t_{i+1} - t_i)}{\sum_{i=0}^{n-1} t_{i+1} - t_i} \tag{1}$$

where s_i is the spread during the time interval $[t_{i+1}, t_i)$.
This spread is only calculated when the ask queue and bid queue are not empty. In case one of them is empty, this time period is not included in the calculation.
Lower spread usually indicates higher trading activity. It can be interpreted as the competition being higher, therefore it is easier to sell and buy everything closer to its market prize.
Since a limit price is drawn from

$$p_n = 100 + 2sin(n \cdot e) \tag{2}$$

The range of the price is [98,102], which means a spread of 1.7 close to 2 shows the best bid and best ask are having a large difference between each other. The trades in this market will be time consuming and easy to be cancelled.

6.2 Average time-to-fill

For each limit order we record the time when it arrived and at the moment of match we calculate how long it took to match. At the end of the simulation we calculate the average of these times. We do not record this time for the orders that got cancelled.
The cancellation time of limit orders are

$$\tau_n = t_{arrival,n} + 30 \cdot (1 + cos^2(n)) \tag{3}$$

which means the duration an order could stay waiting to be matched before being cancelled is in the range [30, 60]. Compared to this, the average fill time of 12.7 seconds is significantly short.

$$\Delta t_n = 5 \cdot (1 + |sin(n\pi/3)|) \tag{4}$$

The range of the inter-arrival time is [5,10], which means an order usually can be matched and filled within 1 or 2 arrivals.

6.3 Cancellation rate

Every time an limit order gets cancelled we update a cancellation counter. Together with limit counter that gets updated every time there is an limit order, we at the end of the simulation calculate the rate of cancelled limit orders over all limit orders. We use the function increment to keep track of the numbers, such that we can keep the counters as objects.
If the cancellation rate is high that means that a big part of the orders got cancelled. That means that no order was compatible at that time.
In our simulation the cancellation rate is around 29%. This is relatively high number and if trading in such scenario, it would be more desirible to have it lower.

6.4 Average queue length

With each new arrival or cancellation we record the queue length. We calculate the average over these values to get the average length of the queue. If the queue length is low that can mean that either the orders got matched fast, or that the cancellation fired before more other orders arrived.
The both queues' depth are about 1, which means the orders are matched quite quickly. The ask queue is slightly longer than the bid queue, meaning that ask orders will accumulate a bit more in the queue.

7 Appendix

7.1 AI statement

We did not use any generative AI for this assignment.